simply need the value of each element, so we use `as $val`. On each iteration the current element value is made available within the loop (via the `$val` variable), which we then use to increment the value of `$total`.

In the case of the `$album` associative array, the keys are also useful metadata. We use the `as $key => $val` form. If we had used the `as $key => $val` form for the `$numbers` loop, then each iteration would have set `$key` to the current index value, `0`, `1`, `2`, and so on. Notice that `$k` and `$v` retain the final values they were assigned by the `foreach` loop. Variables declared within `foreach` expressions persist within the scope that they're defined, even after the loop is complete, so be careful with their names!

Counting Elements

When traversing arrays, you might need to know how many elements an array contains. The built-in function `count()` will return an integer value count of the elements. For multidimensional arrays, `count()` can also make a count of sub-elements by passing the parameter `COUNT_RECURSIVE`:

```
$myArr = ['Alice', 'Bob', 'Russell'];
$i = count($myArr);

$multiDArr = [
    ['Alice', 'Rita'],
    ['Sue', 'Bob'],
];
$j = count($multiDArr, COUNT_RECURSIVE);

print "There are {$i} people in \$myArr\n";
print "There are {$j} people in \$multiDArr\n";

Output:
There are 3 people in $myArr
There are 6 people in $multiDArr
```

`count()` works with countable data types, and arrays are inherently countable, so the return value is as you'd expect: the number of elements in the array in `$myArr` is obviously three. The recursive count of elements in `$multiDArr` is six, which might seem incorrect, but don't forget that the sub-arrays are also elements: we have four strings distributed across two sub-arrays, so the total is six.